Projet de Calcul Parallèle

Parallélisation du Jeu de la Vie de Conway avec MPI

GAO Weihong
CHEN Huixin

30 mars 2026

Table des matières

1 Introduction **2**

2 Architecture du programme **2**
2.1 Séparation affichage / calcul (maître–esclave) 2
2.2 Split du communicateur 2
2.3 Mode benchmark 2

3 Décomposition de domaine **3**
3.1 Décomposition par lignes 3
3.2 Décomposition par colonnes 3
3.3 Décomposition en blocs 2D 3

4 Calcul des voisins **4**

5 Résultats des tests **4**
5.1 Test 1 : Correction 4
5.2 Test 2 : Performance et accélération (*speedup*) 4
5.3 Test 3 : Comparaison des stratégies de décomposition 5
5.4 Test 4 : Extensibilité (*scalability*) 5
5.4.1 Extensibilité forte (*Strong Scaling*) 5
5.4.2 Extensibilité faible (*Weak Scaling*) 6
5.5 Test 5 : Communication vs Calcul 6

6 Structure des fichiers **7**

7 Conclusion **7**

1 Introduction

Le Jeu de la Vie est un automate cellulaire inventé par John Conway en 1970. Il se déroule sur une grille bidimensionnelle où chaque cellule peut être dans l'un des deux états : **vivante** ou **morte**. L'évolution de la grille suit quatre règles simples basées sur le nombre de voisins vivants de chaque cellule :

1. Une cellule vivante avec moins de 2 voisines vivantes meurt (sous-population).
2. Une cellule vivante avec 2 ou 3 voisines vivantes survit.
3. Une cellule vivante avec plus de 3 voisines vivantes meurt (sur-population).
4. Une cellule morte avec exactement 3 voisines vivantes naît (reproduction).

Dans ce projet, la grille est traitée comme un **tore** (topologie torique) : les cellules du bord supérieur sont voisines de celles du bord inférieur, et celles du bord gauche sont voisines de celles du bord droit.

L'objectif est de paralléliser ce calcul en utilisant **MPI** (Message Passing Interface) avec trois stratégies de décomposition de domaine, puis d'analyser les performances.

2 Architecture du programme

2.1 Séparation affichage / calcul (maître–esclave)

Le programme suit une architecture maître–esclave :

— **Rang 0** (groupe A) : processus d'affichage avec `pygame`. Il reçoit les données calculées par les workers et met à jour la fenêtre graphique.

— **Rangs 1 à N-1** (groupe B) : processus de calcul (workers). Chaque worker est responsable d'une portion de la grille.

2.2 Split du communicateur

La séparation des groupes est réalisée avec `MPI.COMM_WORLD.Split()` :

```
color = 0 if rank == 0 else 1
comm_local = comm.Split(color, rank)
```

Le rang 0 reçoit la couleur 0 (groupe A, seul), tandis que les rangs 1 à N-1 reçoivent la couleur 1 (groupe B). Cela permet aux workers de communiquer librement entre eux via leur communicateur local.

2.3 Mode benchmark

Chaque script MPI supporte un mode `-benchmark` qui désactive `pygame` et mesure séparément les temps de calcul et de communication avec `MPI.Wtime()` :

```
mpirun -n 5 python game_of_life_mpi_row.py --benchmark 1000 1000 100 42
```

La sortie produit : `TIME_COMPUTE`, `TIME_COMM`, `TIME_TOTAL`.

3 Décomposition de domaine

Trois stratégies classiques de décomposition ont été implémentées. Dans tous les cas, des **cellules fantômes** (*ghost cells*) sont utilisées pour échanger les données de bordure entre processus voisins.

3.1 Décomposition par lignes

```
+---------------------------+
|  Worker 0 (lignes 0..k)   |  <- ghost row (from Worker N-1)
|---------------------------|
|  Worker 1 (lignes k+1..m) |
|---------------------------|
|  ...                      |
|---------------------------|
|  Worker N-1               |  <- ghost row (from Worker 0)
+---------------------------+
```

FIGURE 1 – Décomposition par lignes : chaque worker reçoit un bloc de lignes contiguës.

La grille de taille $n_y \times n_x$ est découpée horizontalement en W bandes (où $W = \text{size} - 1$ est le nombre de workers). Chaque worker reçoit $\lfloor n_y/W \rfloor$ ou $\lceil n_y/W \rceil$ lignes.

Échanges de ghost cells : Chaque worker échange sa première et dernière ligne avec ses voisins torique (haut et bas) via `Sendrecv`. Les colonnes fantômes gauche/droite sont gérées localement par chaque worker (wrap torique sur les colonnes).

Le tableau paddé a la taille $(n_y^{\text{local}} + 2) \times (n_x + 2)$.

3.2 Décomposition par colonnes

La grille est découpée verticalement. Chaque worker reçoit un bloc de colonnes contiguës.

Échanges de ghost cells : Chaque worker échange sa première et dernière colonne avec ses voisins torique (gauche et droite) via `Sendrecv`. Les lignes fantômes haut/bas sont gérées localement.

Le tableau paddé a la taille $(n_y + 2) \times (n_x^{\text{local}} + 2)$.

3.3 Décomposition en blocs 2D

La grille est découpée en blocs rectangulaires selon les deux dimensions. Le nombre de workers est factorisé en $p_y \times p_x$ via `MPI.Compute_dims`. Une topologie cartésienne torique est créée :

```
dims = MPI.Compute_dims(nb_workers, 2)
cart_comm = compute_comm.Create_cart(dims, periods=[True, True])
```

Échanges de ghost cells (8 échanges par itération) :

— 2 échanges pour les lignes haut/bas (via `cart_comm.Shift(0, 1)`)

— 2 échanges pour les colonnes gauche/droite (via `cart_comm.Shift(1, 1)`)

— 4 échanges pour les coins diagonaux (via `cart_comm.Get_cart_rank`)

Le tableau paddé a la taille $(n_y^{\text{local}} + 2) \times (n_x^{\text{local}} + 2)$.

Avantage théorique : pour une grille carrée de taille $N \times N$, le périmètre du sous-domaine en décomposition 2D est $2(\frac{N}{\sqrt{W}} + \frac{N}{\sqrt{W}}) = \frac{4N}{\sqrt{W}}$, tandis qu'en décomposition par lignes il est $2N$. La communication est donc réduite d'un facteur $\frac{2\sqrt{W}}{1}$ pour les grandes grilles.

4 Calcul des voisins

Le comptage des voisins se fait par décalage de tableaux NumPy sur le tableau paddé :

```
neighbors = (padded[0:-2, 0:-2] + padded[0:-2, 1:-1] + padded[0:-2, 2:]
            + padded[1:-1, 0:-2] +                        padded[1:-1, 2:]
            + padded[2:,   0:-2] + padded[2:,   1:-1] + padded[2:, 2:])
```

Cette approche vectorisée est équivalente à une convolution avec le noyau : $K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}$

Les règles du jeu de la vie sont ensuite appliquées avec `np.where` :

```
new_cells = np.where(local_cells == 1,
                     np.where((neighbors == 2) | (neighbors == 3), 1, 0),
                     np.where(neighbors == 3, 1, 0)).astype(np.uint8)
```

5 Résultats des tests

5.1 Test 1 : Correction

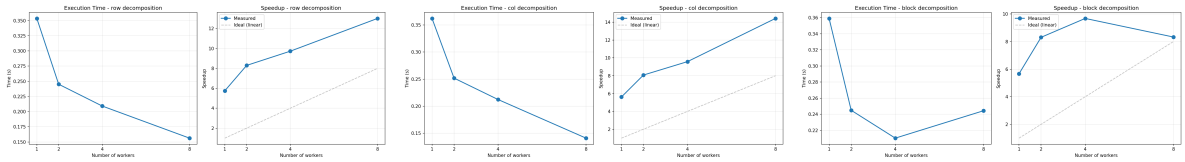
Les trois versions parallèles (lignes, colonnes, blocs) ont été comparées à la référence sérielle sur une grille 50×50 pendant 5 itérations avec une graine aléatoire fixe (`seed=42`).

Stratégie	**Résultat**
Décomposition par lignes	PASS
Décomposition par colonnes	PASS
Décomposition en blocs 2D	PASS

Table 1 – Résultats du test de correction : toutes les grilles parallèles correspondent exactement à la référence sérielle.

5.2 Test 2 : Performance et accélération (*speedup*)

Le speedup est défini par $S = T_{\text{sériel}}/T_{\text{parallèle}}$. Le speedup idéal est égal au nombre de workers.

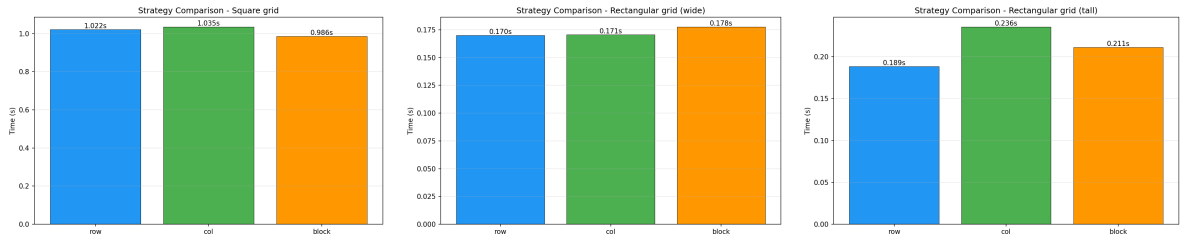


(a) Lignes (b) Colonnes (c) Blocs 2D

FIGURE 2 – Temps d'exécution et speedup pour les trois stratégies de décomposition.

On observe que le speedup dépasse parfois la valeur idéale pour un faible nombre de workers. Cela s'explique par l'utilisation de NumPy vectorisé dans les workers MPI, alors que la référence sérielle utilise `scipy.signal.convolve2d` qui a un overhead plus important. Lorsque le nombre de workers augmente, le coût de communication MPI commence à dominer, réduisant le speedup.

5.3 Test 3 : Comparaison des stratégies de décomposition



(a) Grille carrée (b) Rectangle large (c) Rectangle haut

FIGURE 3 – Comparaison des trois stratégies sur différentes formes de grille (5 processus, 20 itérations).

Analyse :

— Pour une **grille carrée**, la décomposition par lignes et en blocs sont comparables. La décomposition en blocs a l'avantage théorique d'un périmètre minimal, réduisant le volume de communication.

— Pour un **rectangle très large** ($n_y \ll n_x$), la décomposition par **lignes** est plus efficace car chaque worker a peu de lignes à échanger.

— Pour un **rectangle très haut** ($n_y \gg n_x$), la décomposition par **colonnes** devient plus efficace pour la même raison.

5.4 Test 4 : Extensibilité (*scalability*)

5.4.1 Extensibilité forte (*Strong Scaling*)

Le problème a une taille fixe (500×500) et on augmente le nombre de workers.

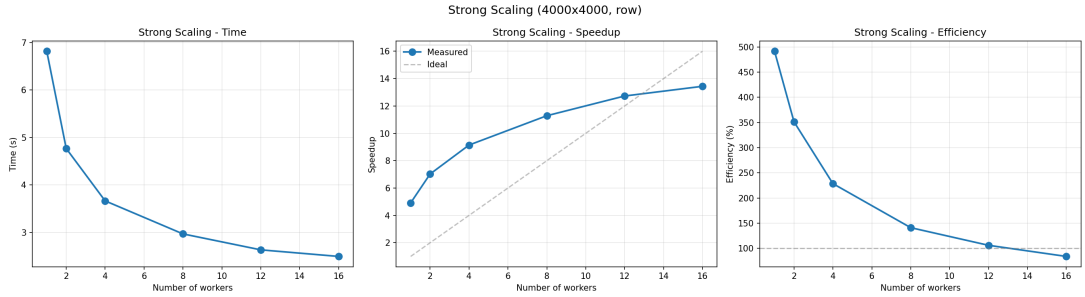


FIGURE 4 – Strong scaling : temps, speedup et efficacité en fonction du nombre de workers.

Analyse : Le temps diminue avec le nombre de workers, mais l'efficacité $\eta = S/W$ diminue également. Cela est dû au fait que lorsque le nombre de workers augmente, la taille du sous-domaine par worker diminue, et le rapport communication/calcul augmente.

5.4.2 Extensibilité faible (*Weak Scaling*)

Chaque worker traite un volume constant de données (200×200 cellules). Le nombre total de cellules augmente proportionnellement au nombre de workers.

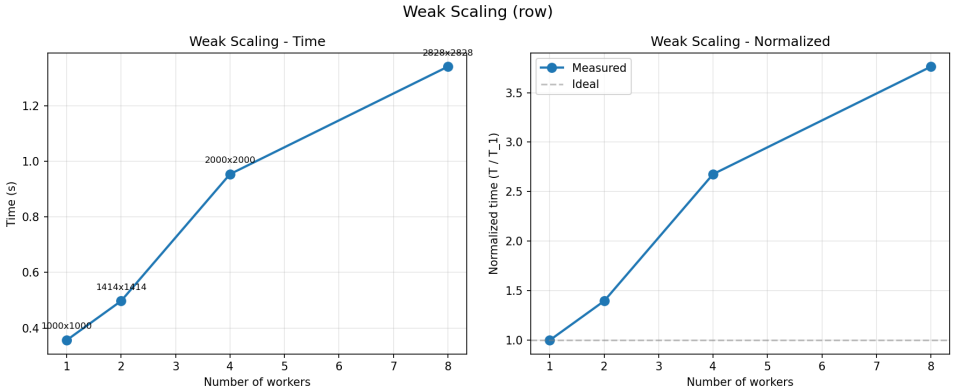


FIGURE 5 – Weak scaling : le temps devrait rester constant si le parallélisme est parfait.

Analyse : En théorie, le temps devrait rester constant. En pratique, on observe une légère augmentation due au coût de communication croissant lorsque le nombre de processus augmente.

5.5 Test 5 : Communication vs Calcul

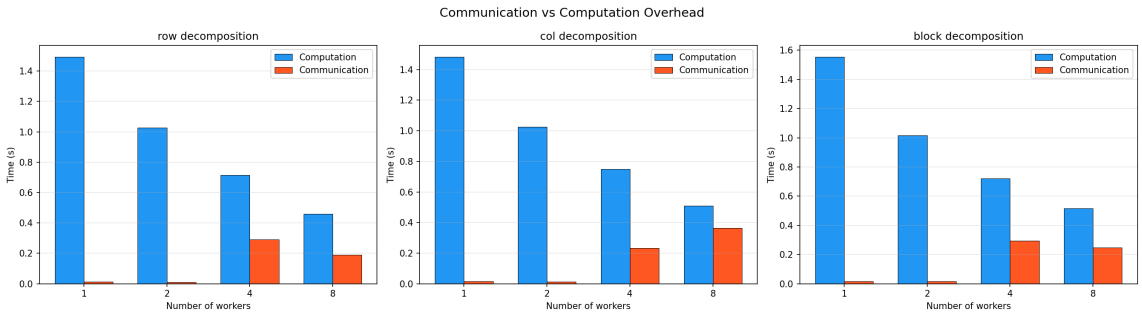


FIGURE 6 – Décomposition du temps en calcul et communication pour chaque stratégie.

Analyse : Quand le nombre de workers augmente :

— Le temps de **calcul** diminue (le travail est partagé).

— Le temps de **communication** augmente (plus d'échanges de ghost cells).

— Au-delà d'un certain nombre de workers, la communication domine et le temps total peut même augmenter. C'est le **goulot d'étranglement de la communication** (*communication bottleneck*).

La décomposition en blocs 2D tend à avoir un ratio communication/calcul plus favorable pour les grandes grilles carrées, car le périmètre de chaque sous-domaine est minimisé.

6 Structure des fichiers

Fichier	Description
`game_of_life.py`	Version sérielle originale (boucles)
`game_of_life_vect.py`	Version sérielle vectorisée (`convolve2d`)
`row.py`	Version MPI – décomposition par lignes
`col.py`	Version MPI – décomposition par colonnes
`block.py`	Version MPI – décomposition en blocs 2D
`benchmark.py`	Suite de tests automatique (5 tests)
`rapport.tex`	Ce rapport

Table 2 – Structure du projet.

7 Conclusion

Ce projet a permis de paralléliser le Jeu de la Vie de Conway avec MPI en utilisant trois stratégies de décomposition de domaine : par lignes, par colonnes et en blocs 2D.

Points clés :

1. La **correction** des trois versions a été vérifiée par comparaison avec la référence sérielle.
2. L'**accélération** est significative pour 2–4 workers, mais se dégrade au-delà en raison du coût de communication.
3. Le **choix de la stratégie** dépend de la forme de la grille : les lignes pour les grilles larges, les colonnes pour les grilles hautes, les blocs 2D pour les grilles carrées.
4. Le **goulot d'étranglement de la communication** est clairement observable : au-delà d'un certain nombre de cores, le temps de communication dépasse le temps de calcul.
5. La **topologie torique** est correctement gérée dans les trois cas grâce aux cellules fantômes (*ghost cells*).